Lance Derick H. De Villa August 2, 2026

Machine Problem #1 Operating Systems

FCFS CPU Scheduling Simulator

Machine Problem #1: Write a Program that Simulates the FCFS CPU Scheduling Algorithm to Find Turnaround and Waiting Time

I. Source Code (Python)

The program below implements the non-preemptive First-Come, First-Served (FCFS) CPU scheduling algorithm in Python. It validates the number of processes (3–10), enforces unique process IDs and unique arrival times, computes each process's completion, turnaround, and waiting time, and displays the average waiting time and average turnaround time.

```
"""
Machine Problem #1
FCFS (First-Come, First-Served) CPU Scheduling Simulator
------------------------------------------------------------
Computes Turn-Around Time and Waiting Time for a set of
processes using the non-preemptive FCFS scheduling algorithm.

Validation rules implemented:
    - Number of processes must be between 3 and 10 (inclusive)
    - Process IDs must be unique
    - Arrival times must be unique
    - Arrival time and burst time must be non-negative integers
"""


def get_valid_int(prompt, min_value=None):
    """Repeatedly prompt until the user enters a valid integer
    that satisfies the optional minimum value constraint."""
    while True:
        raw_value = input(prompt).strip()
        try:
            value = int(raw_value)
        except ValueError:
            print("Invalid input. Please enter a whole number.")
            continue

        if min_value is not None and value < min_value:
            print(f"Value must be {min_value} or greater.")
            continue

        return value


def get_number_of_processes():
    """Ask the user for the number of processes, restricted to 3-10."""
    while True:
        count = get_valid_int("Enter the no. of process: ", min_value=0)
```

```
        if 3 <= count <= 10:
            return count
        print("Number of processes must be between 3 and 10 only.")


def get_process_id(index, existing_ids):
    """Ask for a process ID, ensuring it is non-empty and unique."""
    while True:
        pid = input(f"Enter process ID for Process{index}: ").strip()
        if not pid:
            print("Process ID cannot be empty.")
            continue
        if pid in existing_ids:
            print(f"Process ID '{pid}' is already used. It must be unique.")
            continue
        return pid


def get_arrival_time(pid, existing_arrivals):
    """Ask for an arrival time, ensuring it is unique and non-negative."""
    while True:
        arrival = get_valid_int(f"Enter arrival time for {pid}: ", min_value=0)
        if arrival in existing_arrivals:
            print("Arrival time is already used by another process. "
                  "It must be unique.")
            continue
        return arrival


def get_burst_time(pid):
    """Ask for a burst time, ensuring it is a positive integer."""
    return get_valid_int(f"Enter burst time for {pid}: ", min_value=1)


def collect_processes():
    """Gather process ID, arrival time, and burst time for every process."""
    num_processes = get_number_of_processes()

    processes = []
    used_ids = set()
    used_arrivals = set()

    for i in range(1, num_processes + 1):
        pid = get_process_id(i, used_ids)
        used_ids.add(pid)

        arrival = get_arrival_time(pid, used_arrivals)
        used_arrivals.add(arrival)

        burst = get_burst_time(pid)

        processes.append({
            "pid": pid,
            "arrival": arrival,
            "burst": burst,
        })

    return processes


def compute_fcfs(processes):
```

```
    """Compute completion, turn-around, and waiting time for each
    process using the FCFS (non-preemptive) scheduling algorithm.

    FCFS orders processes by arrival time; the CPU executes each
    process to completion before moving to the next one.
    """
    # Sort a copy of the process list by arrival time (FCFS order)
    ordered = sorted(processes, key=lambda p: p["arrival"])

    current_time = 0
    for process in ordered:
        # If the CPU is idle before this process arrives, jump ahead
        if current_time < process["arrival"]:
            current_time = process["arrival"]

        start_time = current_time
        completion_time = start_time + process["burst"]
        turnaround_time = completion_time - process["arrival"]
        waiting_time = turnaround_time - process["burst"]

        process["start"] = start_time
        process["completion"] = completion_time
        process["turnaround"] = turnaround_time
        process["waiting"] = waiting_time

        current_time = completion_time

    return ordered


def display_results(processes):
    """Print a formatted table of results plus the averages."""
    print("\n" + "=" * 70)
    print(f"{'PID':<8}{'Arrival':<10}{'Burst':<10}"
          f"{'Completion':<13}{'Turnaround':<13}{'Waiting':<10}")
    print("-" * 70)

    total_turnaround = 0
    total_waiting = 0

    for p in processes:
        print(f"{p['pid']:<8}{p['arrival']:<10}{p['burst']:<10}"
              f"{p['completion']:<13}{p['turnaround']:<13}{p['waiting']:<10}")
        total_turnaround += p["turnaround"]
        total_waiting += p["waiting"]

    count = len(processes)
    avg_turnaround = total_turnaround / count
    avg_waiting = total_waiting / count

    print("=" * 70)
    print(f"Average waiting time: {avg_waiting:.2f}")
    print(f"Average turn-around time: {avg_turnaround:.2f}")
    print("=" * 70)


def main():
    print("FCFS CPU Scheduling Simulator")
    print("-" * 32)

    processes = collect_processes()
```

```
    results = compute_fcfs(processes)
    display_results(results)


if __name__ == "__main__":
    main()
```

II. Sample Run / Results

Below is a sample program run using five processes. The program was executed in a terminal; the numbers typed by the user are shown immediately after each prompt.

```
FCFS CPU Scheduling Simulator
-------------------------------
Enter the no. of process: 5
Enter process ID for Process1: P1
Enter arrival time for P1: 0
Enter burst time for P1: 8
Enter process ID for Process2: P2
Enter arrival time for P2: 1
Enter burst time for P2: 4
Enter process ID for Process3: P3
Enter arrival time for P3: 2
Enter burst time for P3: 2
Enter process ID for Process4: P4
Enter arrival time for P4: 3
Enter burst time for P4: 5
Enter process ID for Process5: P5
Enter arrival time for P5: 4
Enter burst time for P5: 7

=========================================================================
PID     Arrival   Burst     Completion   Turnaround   Waiting
-------------------------------------------------------------------------
P1      0         8         8            8            0
P2      1         4         12           11           7
P3      2         2         14           12           10
P4      3         5         19           16           11
P5      4         7         26           22           15
=========================================================================
Average waiting time: 8.60
Average turn-around time: 13.80
=========================================================================
```

Verification (P1, arrival 0, burst 8): the CPU is idle before it arrives, so it starts immediately at time 0 and finishes at time 8, giving a turnaround time of 8 and a waiting time of 0. Each later process must wait for every earlier process in the queue to finish before it can start, which is why P5's waiting time (15) is the largest — it arrived last and had to wait behind P1 through P4.

III. Questions

1. How can you determine which CPU scheduling algorithm is effective?

An algorithm's effectiveness is judged using CPU scheduling criteria rather than a single number. The most common measures are average waiting time and average turnaround time (lower is better, since processes spend less time sitting idle in the ready queue), CPU utilization (how much of the time the CPU is kept busy), throughput (how many processes complete per unit of time), and response time (how quickly an interactive process gets its first slice of CPU time). No single algorithm is best for every situation — FCFS, for instance, is simple and fair in arrival order but can produce long average waiting times when a long process arrives before several short ones (the convoy effect). An algorithm is considered effective for a given workload when it minimizes waiting and turnaround time while keeping CPU utilization and throughput high, relative to the goals of the system (e.g., a batch system favors throughput, while an interactive system favors response time).

2. How do you compute the waiting time and burst time of every process?

Burst time is not computed — it is given/estimated in advance as the amount of CPU time a process needs to finish executing; in this program the user supplies it directly for each process. Waiting time, on the other hand, is derived once the process has actually been scheduled. For a non-preemptive algorithm like FCFS, the steps are:

1. Completion Time = the time the CPU actually finishes running the process (for FCFS: Start Time + Burst Time, where Start Time is either the process's arrival time or the moment the CPU becomes free, whichever is later).
2. Turnaround Time = Completion Time − Arrival Time.
3. Waiting Time = Turnaround Time − Burst Time.

In other words, waiting time is simply the portion of the turnaround time that the process spent sitting in the ready queue instead of running on the CPU. This is exactly what compute_fcfs() in the program above does for each process.

3. How do you compute the average waiting time and average turnaround time of every process?

Once every individual process's waiting time and turnaround time have been computed, the averages are found by summing each value across all processes and dividing by the total number of processes (n):

Average Waiting Time = (Σ Waiting Timei) / n

Average Turnaround Time = (Σ Turnaround Timei) / n

In the program, this is done in display_results(), which accumulates total_waiting and total_turnaround while printing each row, then divides both totals by the number of processes to print the two averages shown in the sample run above (Average waiting time: 8.60, Average turn-around time: 13.80).

IV. Conclusion

This machine problem demonstrated how the First-Come, First-Served scheduling algorithm assigns the CPU to processes strictly in the order they arrive, with no preemption once a process begins execution. Implementing it in Python reinforced how turnaround time and waiting time are derived from just two inputs — arrival time and burst time — and how input validation (unique IDs, unique arrival times, a bounded process count) keeps the simulation well-defined. The sample run also illustrated FCFS's main weakness, the convoy effect: because scheduling order depends only on arrival time and not on burst length, a process that arrives early but has a long burst time can push back the start time — and therefore inflate the waiting time — of every process that arrives after it, even short ones. This makes FCFS simple and easy to implement, but not necessarily optimal for minimizing average waiting time compared to algorithms like Shortest Job First.